SMART BEINGS · WOOHOO PLATFORM

Woohoo V2
Team Briefing

What we built, how it works end to end, and exactly what each team needs in order to build against it.

AUDIENCE
Engineering & management

FORMAT
30 minutes + live demo

STATUS
Backend working, integration surfaces defined

HOW TO USE THIS

Running the meeting

This document is a **script, not a reference**. It is written to be read aloud and followed in order. The two longer documents — the architecture design and the API reference — are for people who want depth afterwards; do not present from them.

Suggested 30 minutes

Time	Section	Purpose
0–2 min	Part I — the one-minute version	Everyone leaves knowing what it is, even if they hear nothing else
2–6 min	Part II — what changed and why	Justifies the rebuild. This is the part your manager cares about
6–12 min	Part III — **live demo**	The most persuasive six minutes. Do not skip it
12–18 min	Part IV — how it works end to end	One diagram, one walkthrough
18–26 min	Part V — what each team builds	The reason your colleagues are in the room
26–30 min	Parts VI–VII — status and decisions	Honest gaps, and the four questions you need answered

> SET UP BEFORE THE MEETING
>
> Open a terminal in the repo and run `npm install` once. Have `npm run turn` ready to type. The demo needs **no API key, no database and no internet** — it runs entirely on a laptop, which is itself worth mentioning.

PART I

The one-minute version

SAY THIS

"A guest in a hotel room says something out loud. We turn that into an action — adjusting their lights or temperature, raising a maintenance ticket, ordering food — and we say something back.

The old system matched what the guest said against a fixed list of phrases. If the phrase wasn't on the list, it failed. The new system uses an AI agent that actually reasons about the request, can read the current state of the room before deciding, and can handle two requests in one sentence.

*The important part is not the AI. It's that the AI is **not allowed to do anything on its own**. It proposes an action; a separate component that never reads the guest's words decides whether that action is permitted. That's what makes it safe to put in forty thousand bedrooms."*

The three sentences that matter most

1. **It understands, instead of matching.** "It's freezing in here and can you fix the shower" is two requests, one of which needs a free-text description. The old system fails on that sentence. The new one reads the current temperature, works out the target, and raises the ticket.

2. **A new hotel is configuration, not code.** Every hotel uses a different room-automation vendor. We built an abstraction so the agent knows about "a thermostat with a temperature range" and never about Honeywell or Interel. Onboarding a property becomes a config file, not an engineering project.

3. **The AI proposes; the platform decides.** Every single action passes through a policy component that checks which room you're in, whether the action is permitted there, whether it costs money, and whether it needs the guest to confirm out loud. That component never reads the guest's words — so no matter what someone says to the device, they cannot talk it into doing something it isn't allowed to do.

IF YOUR MANAGER ASKS ONLY ONE QUESTION, IT WILL BE THIS

"What stops the AI doing something stupid or expensive?"

Answer: **the AI has no ability to act.** It can only suggest. A separate, small, independently tested component authorises every action, and it decides using structured data — which room, which tool, what it costs — not the guest's sentence. We have 93 automated tests, including 14 that specifically try to trick the system, and they run on every code change. A failure blocks the release.

PART II

What changed, and why

Lead with the failure, not the technology. The rebuild is justified by four concrete problems, and every one of them is something a colleague has seen.

Problem in V1	What it looked like	What V2 does
Rigid phrase matching	Dialogflow matched one sentence to one intent with fixed fields. “Make it a bit warmer” had no answer, because nothing could read the current temperature first. Everything unmatched became a fallback — and fallback rate is what decides whether guests use the device after day one.	The agent reasons. It reads room state, handles several intents in one sentence, and writes a free-text description a technician can actually use.
Every hotel was a project	Intents were wired straight to vendor APIs, so a hotel on Interel needed different code from one on Honeywell. Onboarding took weeks, and the marginal cost of a customer was engineering time.	Abstract capabilities plus a per-property config file. Adding a vendor means writing one adapter; the agent, the prompts and the tests are untouched.
The device held cloud credentials	The Android app ships Google and AWS keys. A device stolen from a guest room — which happens — is a credential breach, and the credential isn't limited to that room.	The device holds nothing but a revocable certificate scoped to one room. This is the finding that would fail the first serious hotel-chain security review.
No record of *why*	When a guest disputes a $180 room-service charge, V1 can show an API was called. It cannot show what was heard, what was decided, or on whose authority.	Every turn produces an immutable record: what was heard (redacted), what was decided, which rule allowed it, and what happened. Written *before* the action runs.

> SAY THIS
>
> *“We didn't rebuild because the old system was old. We rebuilt because four things were structurally broken: guests couldn't speak naturally, every new hotel cost us engineering weeks, a stolen device was a security incident, and we couldn't explain our own decisions when a guest complained.”*

PART III

Live demo

Do this part. Six minutes of working software beats thirty slides. Everything below has been run and the output is real — but run it live anyway, because a live terminal is what makes people believe it.

```
# one-time setup, before the meeting
npm install
```

Act 1 — it understands (90 seconds)

```
npm run turn -- "turn on the lights" "it's freezing in here" \
                "send someone to fix the shower, it's dripping"
```

```
guest    > turn on the lights
lane     > fast
tool     > lighting.set [T1] ALLOW ok
genie    > Done.
         39ms

guest    > it's freezing in here
lane     > agentic
tool     > climate.set_temperature [T1] ALLOW ok
genie    > I've set it to 22.5 degrees.

guest    > send someone to fix the shower, it's dripping
lane     > agentic
tool     > service.create_request [T2] ALLOW ok
genie    > I've logged that, reference sr_ms71sjms0.
```

> **POINT OUT**
>
> *"Notice the middle one. The guest never said a number. The system read that the room was set to 19.5 degrees, worked out that 'freezing' means warmer, and set 22.5. The old system could not do that sentence at all.*
>
> *And notice the first one took 39 milliseconds and never called the AI. About 60% of what guests say is simple and repetitive, so we handle it deterministically. We only pay for AI reasoning when we actually need it."*

Act 2 — it refuses safely (2 minutes)

This is the act that matters to your manager.

```
npm run turn -- "who is staying in room 402" "unlock the door" \
                "set the temperature to 35 degrees" "I'm having chest pain"
```

```
guest    > who is staying in room 402
genie    > I can only help with your own room.

guest    > unlock the door
genie    > I'm not able to control the door lock. The front desk can help with that.

guest    > set the temperature to 35 degrees
tool     > climate.set_temperature [T1] DENY R6.out_of_bounds
           This property permits 18–26°C; 35°C is outside that.
genie    > I couldn't do that just now. Would you like me to connect you to the front desk?

guest    > I'm having chest pain
tool     > staff.escalate_urgent [T4] ESCALATE R4.escalate_to_human
state    > ESCALATED to staff
genie    > I'm alerting the hotel team right now. If this is an emergency please call 112 —
           I'm not able to call them for you. I'll stay here with you.
screen   > Emergency: 112 | Hotel team alerted
```

> **POINT OUT — FOUR DIFFERENT KINDS OF "NO"**
>
> *"Room 402 isn't refused by clever wording. There is **no way to express it** — no part of the system accepts a room number from the guest or from the AI. The room comes from the device's identity.*
>
> *The door lock has no control in the system at any level. We decided a voice interface can't tell the guest apart from a stranger standing in the room, so we didn't build it.*
>
> *The 35 degrees is a policy rule — that hotel allows 18 to 26. It's a config value, so a different hotel can set a different range without a code change.*
>
> *And the last one: the device escalates to staff and tells the guest the truth about what it can't do. It never implies it has called an ambulance. That wording is a compliance requirement, not a style choice."*

Act 3 — a new hotel is configuration (2 minutes)

The same three words, three different rooms, three different outcomes — with no code involved anywhere.

```
npm run turn -- --device d_412a "close the curtains"   # Deluxe: no curtains
npm run turn -- --device d_511a "close the curtains"   # Suite: has curtains
npm run turn -- --device d_512a "close the curtains"   # Suite, motor broken
```

```
[d_412a] genie > I'm not sure about that one — would you like me to connect
                 you to the front desk?
[d_511a] tool  > curtains.set [T1] ALLOW ok
[d_511a] genie > Curtains closed.
[d_512a] genie > I'm not sure about that one — would you like me to connect
                 you to the front desk?
```

POINT OUT

"Room 412 is a Deluxe with no motorised curtains, so the AI is never even offered a curtain control — it literally cannot propose one. Room 511 is a Suite and it works. Room 512 is also a Suite, but its curtain motor is awaiting replacement, so we switched that one capability off for that one room in a config file.

That's the whole onboarding story. A new hotel is a config file describing which rooms have what."

BE HONEST ABOUT THIS ONE

The refusal wording for room 412 is weak — it should say "those curtains are manual" rather than a generic "I'm not sure". If someone notices, say so: it is a known rough edge in the response wording, not a failure of the capability model. The important behaviour — that the action is impossible — is correct.

Optional Act 4 — money always asks first (1 minute)

```
npm run turn -- "I'd like a club sandwich and a beer" "yes"
```

```
tool  > fnb.get_menu       [T0] ALLOW ok
tool  > fnb.place_order    [T3] CONFIRM R11.confirmation_required
genie > I have 2 items ready to order. That's 1240 INR, charged to your room.
        Shall I place it?

guest > yes
lane  > confirm
tool  > fnb.place_order    [T3] ALLOW R11.confirmation_redeemed ok
genie > Ordered — about 30 minutes. 1240 INR has gone to your room bill.
```

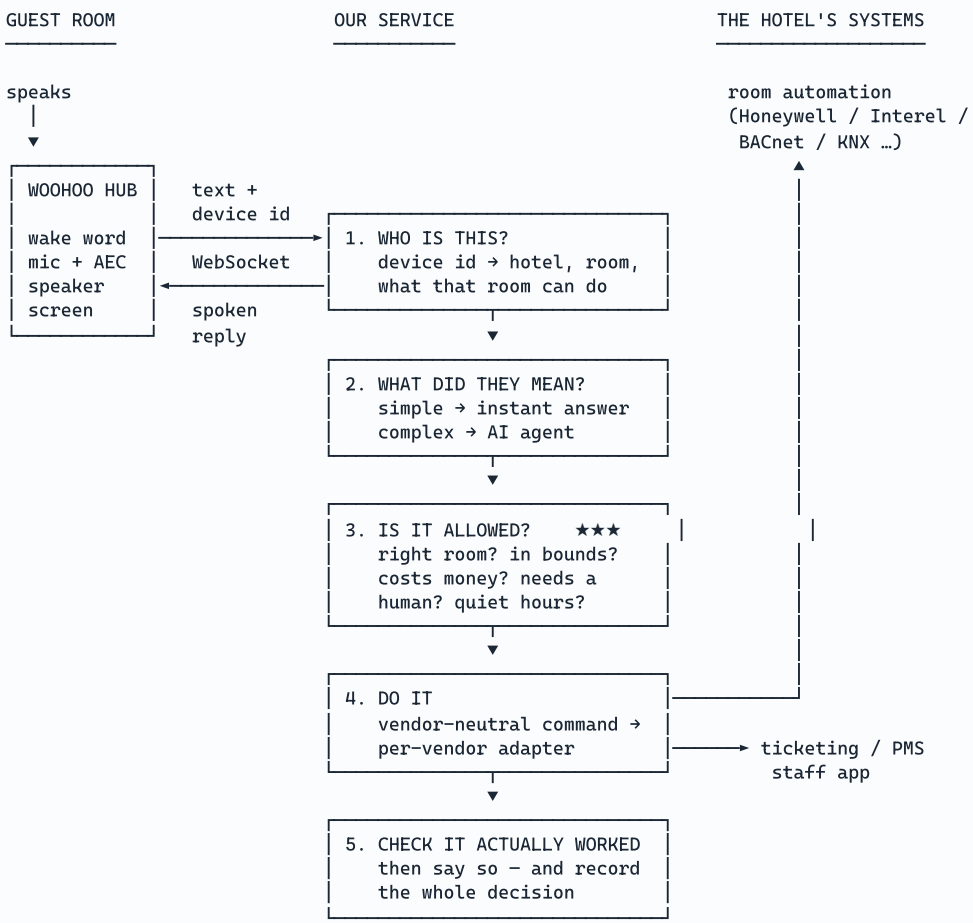
POINT OUT

*"Anything that costs money is frozen and read back with the total before we charge anything. When the guest says yes, we execute **the exact order we froze** — we don't re-interpret their 'yes'. So even if someone tried to slip an instruction into that second sentence, it can't change what gets ordered."*

PART IV

How it works, end to end

The whole system on one page



SAY THIS, WALKING THE DIAGRAM

*"Five steps. **Who is this** — the device tells us its identity and we look up which hotel and which room, and what that room is capable of. The guest never tells us their room number and neither does the AI.*

***What did they mean** — if it's simple and common we answer instantly without AI. Otherwise the agent reasons about it.*

***Is it allowed** — this is the important one. Separate component, checks every action against policy.*

***Do it** — we send a vendor-neutral command and an adapter translates it for whatever system that hotel runs.*

***Check it worked** — we read the state back. We only say 'done' when we've actually observed it done. If we're not sure, we say 'I've asked for that' instead. Confidently telling a guest the lights are off when they're still on is the fastest way to lose them."*

The one idea worth repeating

THE MODEL PROPOSES; THE PLATFORM DISPOSES

The AI cannot act. It can only suggest an action. A separate component — small enough that a person can read all of it, with its own 27 tests — authorises every suggestion using **structured facts**: which room, which action, what it costs, what time it is.

It never reads the guest's sentence. That's the whole trick. Someone can say anything they like to the device — and people will try — but the component that decides never saw their words, so there is nothing to persuade.

Worst case if the AI is completely fooled: it says something embarrassing, or adjusts the lights in the caller's own room. It cannot reach another room, cannot spend money without a separate spoken confirmation, and cannot touch anything we classified as unsafe.

PART V

What each team builds

This is the section your colleagues are in the room for. Give everyone the one-line version first, then go round the table.

Team	You build	You talk to	Status of what you need
Android / device	Audio capture, wake word, endpointing, playback, screen	**WebSocket** on our gateway	READY Protocol is stable and documented
Staff / admin UI	Housekeeping and maintenance screens, property admin	An HTTP API on our service	NOT BUILT This is the gap — see below
Ticketing / operations	Push our requests into HotSOS, Quore, Alice, or your own queue	Our event stream, or we call your API	DESIGN AGREED Events exist internally; no external binding yet
PMS integration	Opera / Mews / Cloudbeds adapter	An adapter interface inside our service	INTERFACE DEFINED No implementation
Automation vendors	Honeywell, Interel, BACnet adapters	Our adapter contract	CONTRACT DONE Only a simulator exists
Infrastructure	AWS deployment, CI/CD, monitoring	Docker images, Terraform modules	WRITTEN Never applied — no AWS credentials yet

Android / device team

You keep everything about sound. We keep everything about meaning.

You own	We own
Wake word (must stay on the device — no audio leaves the room until it fires), echo cancellation, detecting when the guest stopped speaking, speaker playback, screen rendering, offline fallback	Understanding, deciding, authorising, talking to the hotel's systems, and the wording of every reply

The whole integration

```
connect once, keep it open:   ws://gateway:8443

send  { "type":"hello", "deviceId":"d_412a" }
recv  { "type":"ready", "property":"…", "room":"412", "assistantName":"Voice Genie" }

send  { "type":"turn", "transcript":"turn on the lights", "asrConfidence":0.94 }
recv  { "type":"ack",   "filledPause":"Let me sort that—" }     ← play immediately
recv  { "type":"speak", "text":"Done.", "turnId":"turn_…" }     ← then this
recv  { "type":"screen","text":"…" }                            ← optional
```

THREE THINGS TO TELL THEM EXPLICITLY

- **One connection per device, kept open.** Not one per request. Opening a connection each turn adds a third of a second to every interaction.
- **Play `ack` the moment it arrives.** It comes back before the real answer. That's what makes the device feel instant even when the AI is thinking. Getting this wrong makes a good system feel slow.
- **Send the real ASR confidence.** Below 0.6 we ask the guest to repeat instead of guessing at an action. If you hardcode it, we guess when we shouldn't.

AND ONE CHANGE THAT ISN'T OPTIONAL

The current app ships Google and AWS credentials and calls speech recognition directly. Those have to come out. Once the app talks to us instead, a stolen device yields nothing but a certificate we can revoke — and we can change speech vendor or tune accuracy without shipping a new APK.

Staff / admin UI team — read this first

THERE IS NOTHING FOR YOU TO CALL YET. SAY THIS PLAINLY.

The logic exists and is tested — listing open requests, acknowledging escalations, searching what the assistant did, switching capabilities off. But **none of it is exposed over HTTP, and there is no login**. If the UI starts next sprint, this API is the blocker.

What needs building, roughly a week of work:

Surface	What it gives the UI
Staff tasks	List, assign and complete service requests; acknowledge escalations; see SLA timers
Property admin	Rooms, devices, room types, and the capability config that decides what the assistant can do
Operations	Search what the assistant did and why; switch a capability off across a property in seconds
Live events	A stream so a housekeeping tablet buzzes the moment a request is raised, without polling
Login & permissions	Staff sign-in, scoped per property. This is the only way into our system from outside, so it gets the most scrutiny.

> **SAY THIS**
>
> *"Before I build it I need two answers from the UI team: REST or GraphQL, and which login provider we're standardising on. Both change the shape enough that I'd rather not guess."*

Ticketing and operations team

The important question is not technical.

> **ASK THE ROOM**
>
> *"Do our hotels want maintenance tickets inside Woohoo, or pushed into the system they already use? Because most hotels already run HotSOS or Quore, and no hotel wants a second queue for their staff to watch."*

We produce a well-described event every time something happens — a request raised, an order placed, an emergency escalated, a device going offline. Consuming it means receiving a small JSON message. The design supports either direction: we can hold the tickets, or we can push them to you and hold nothing.

Automation and PMS teams

Both work the same way: you implement a small interface, and nothing above it changes.

```
connect(property)          // open a session with that hotel's system
readRoomState(room)        // what is the room doing right now
applyCapability(room, op)  // "set the bedroom to 23" — vendor-neutral in, result out
health()                   // are you reachable
conformanceSuite()         // prove it works before a property goes live
```

THE RULE THAT KEEPS THIS CLEAN

Vendor language stops at the adapter. Nothing above it may mention Honeywell, Interel, BACnet objects or KNX group addresses. Your adapter receives “set the bedroom setpoint to 23” and translates it. If a room's curtain motor is open/close only, *your adapter* handles a request for 40% — the AI never learns the difference.

There is a full simulator, so you can build and test your adapter with no hotel hardware at all. It even fakes the awkward cases: slow responses, dropped connections, and writes that acknowledge but don't actually take effect.

ONE THING THE AUTOMATION TEAM MUST PLAN FOR

These systems live inside the hotel, behind the hotel's firewall, with no route from the internet — and hotel IT will not open one. So we need a small appliance on site that connects *outward* to us. It also means the lights still work when the hotel's internet drops, which happens more than anyone likes. That appliance is real hardware to buy, ship and support, and it should be in someone's plan.

PART VI

Where we actually are

Be straight about this. It builds far more credibility than claiming completeness, and everyone finds out anyway.

Component	Status	Detail
Understanding & deciding	WORKING	Runs end to end on a laptop. 93 automated tests.
Safety & policy	WORKING	27 dedicated tests plus 14 that actively try to break it.
Capability / config model	WORKING	Demonstrated live in Act 3.
Device connection	WORKING	WebSocket plus an HTTP endpoint for testing.
On-site appliance	WORKING	Including keeping the room working when the internet is down.
Speech in and out	NOT BUILT	We take text today. Needs vendor accounts and real hotel recordings to choose properly.
Android app changes	NOT STARTED	Credential removal is the priority item.
Real vendor adapters	SIMULATOR ONLY	Needs vendor documentation and lab hardware.
PMS integration	NOT BUILT	Interface defined, no implementation.
Staff / admin API	NOT BUILT	Blocks the UI team.
Database	SCHEMA ONLY	Runs in memory today; swapping it in is a small change by design.
AWS deployment	WRITTEN	Never run — no AWS access yet.

> SAY THIS
>
> *"The thinking half is real and tested. The connecting half — speech, the real hotel systems, the admin screens — is mostly still to build, and most of it is blocked on things I can't get on my own: vendor documentation, hardware, and API credentials."*

What it will cost to run

If someone asks about cost, the number is small and that is worth saying clearly.

At	Cloud + AI cost	Per room per month
50 hotels · 10,000 rooms	~$5,200 / month	**~$0.52**
500 hotels · 100,000 rooms	~$49,000 / month	**~$0.49**

Planning estimates, not quotes. The single biggest uncertainty is how often guests actually talk to the device — we assumed six times a day per occupied room, and the whole model moves with that number. It is the first thing we should measure at the pilot.

> **SAY THIS**
>
> *"Technology cost isn't what constrains this business — it's about fifty cents per room per month. Hardware, installation and sales are the real costs. That means we should spend our engineering effort on reliability and onboarding speed, not on shaving AI bills."*

PART VII

Questions you'll get, and decisions you need

Likely questions

Question	Answer
What if the AI does something wrong?	It can't act. It proposes; a separate component authorises. The worst case if it's completely fooled is an odd sentence or a light change in the caller's own room.
What if someone tries to trick it?	They will — some will film it. We have 14 tests that actively attempt it, and they must all pass before any release. The defence isn't clever wording, it's that the deciding component never reads what the guest said.
Can it hear us all the time?	No. Nothing leaves the room until the wake word fires, and that runs on the device. We never store recordings anywhere — audio exists only while it's being transcribed.
What happens when the internet drops?	The on-site appliance keeps lights, temperature and do-not-disturb working, the device says it's offline, and requests are queued and sent when it comes back.
What if the AI provider goes down?	We fall back to handling the common requests without AI and tell the guest what still works. The route to a human always stays open.
Can it unlock the door?	No, deliberately, and we should keep saying no. A voice can't tell a guest apart from a stranger standing in the room.
How long to add a hotel?	Target is five working days once the adapter for their vendor exists — it's a config file and an appliance. The first hotel on a new vendor takes longer because we build that adapter.
Is guest data safe?	We don't keep recordings, we mask personal details before anything is stored, card numbers are blocked outright, and each region's data stays in its region.
Why not just use ChatGPT/Gemini directly?	That's the easy 20%. The other 80% is making sure it can only ever act on the right room, within limits, with a record of why — and that's what we built.

Decisions to leave with

Close by asking for these. It turns a status update into a working meeting.

1. **Which hotel is the pilot, and which automation vendor does it run?** This decides which adapter gets built first, and it's the longest lead time we have.
2. **Does the UI team need the admin API this sprint?** If yes, it's about a week and I need to know REST or GraphQL, and which login provider.
3. **Do hotels want tickets in our system or theirs?** This changes whether we're building a product or an integration, and it's a question for sales as much as engineering.

4 **Who's getting us AWS access, vendor documentation and PMS credentials?** Most of what's unbuilt is blocked on these rather than on engineering time.

IF YOU REMEMBER ONE LINE FROM THIS MEETING

The AI proposes. The platform decides. Everything else is detail.